

Assignment 3

Q1) create two networks each of with its own subnet, create a router and connect both subnet to it

Launch instances on both network and verify the topology
As In the Question Firstly, created the Instance named HPC

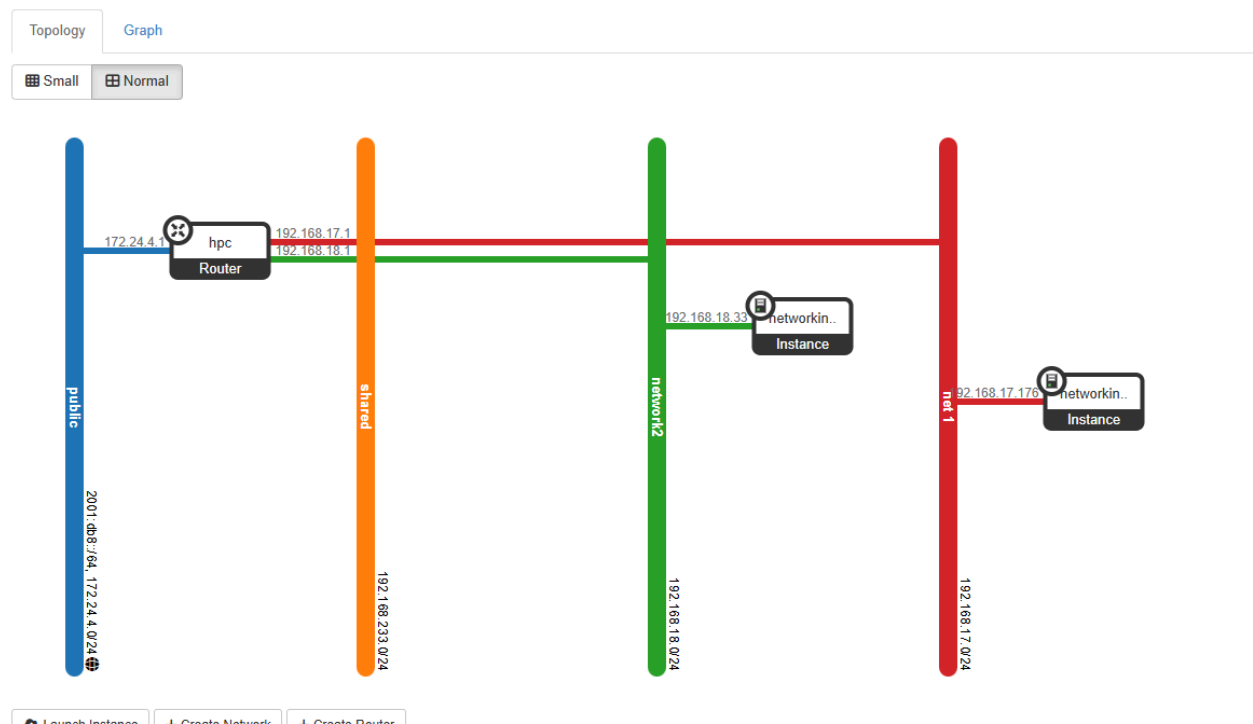
Then two private network created net1, network 2

Then Create the router ,go to the interface of the Router

Join to the public network then followed by net1 and network 2

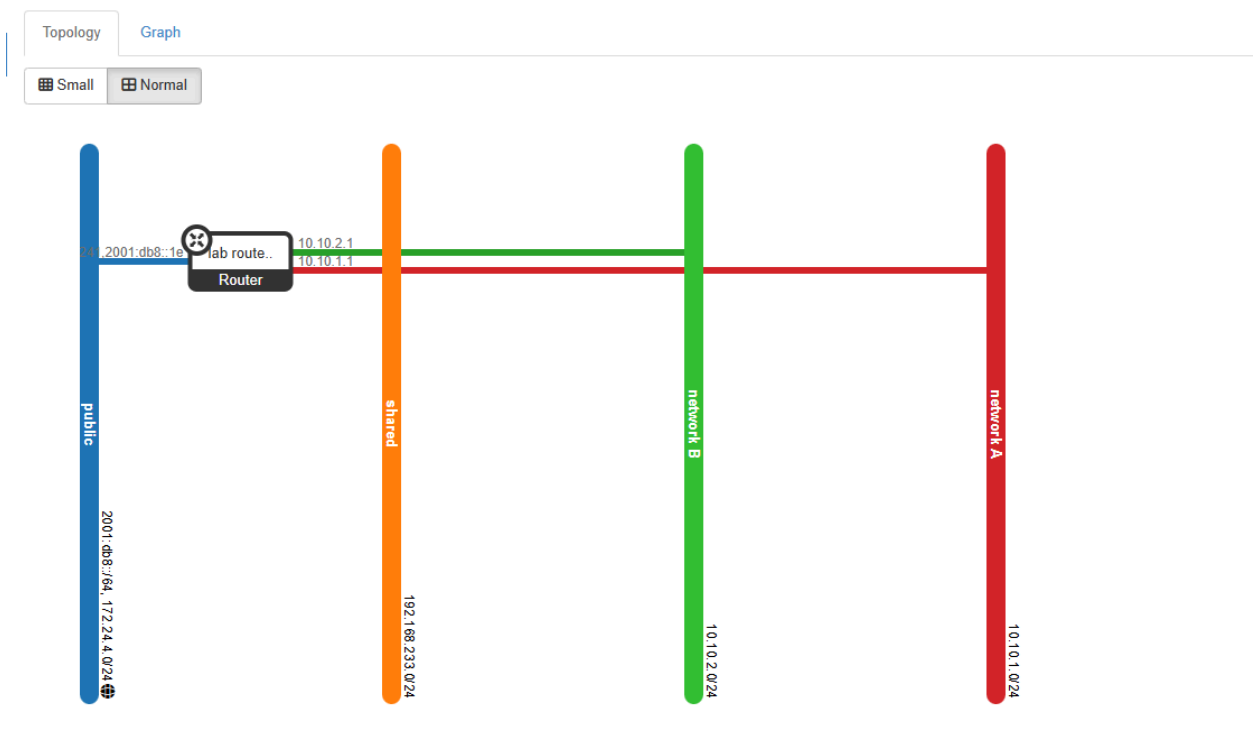
Join the two different Instances to two Different Networks.

Network Topology



Q2) create network A with address 10.10.1.0/24 network B with address 10.10.2.0/24 , router name LabRouter connects both networks to the router and verify the topology

Network Topology



In the Question Router is Connected to the Public Network and another two Private Network

Q3) create two instances with different flavour resize the first instance to match second cinder flower and verify the instance status before and after resizing

flavourinst2

Overview	Interfaces	Log	Console	Action Log
----------	------------	-----	---------	------------

Name	flavourinst2
ID	d22cc433-4ec4-4b87-91b2-59363e6ab1db
Description	-
Project ID	55dc92c4ddfd4ba6b4c99b5353d7fd51
Status	Active
Locked	No
Availability Zone	nova
Created	Sept. 11, 2026, 1:24 p.m.
Age	3 minutes
Host	harsh
Instance Name	instance-0000001b
Reservation ID	r-yhjkv6s1
Launch Index	-
Hostname	flavourinst2
Kernel ID	-
Ramdisk ID	-
Device Name	/dev/vda
User Data	-

Specs

Flavor Name	m1.micro
Flavor ID	84
RAM	256MB
vCPUs	1 vCPU
Disk	1GB

IP Addresses

public	172.24.4.2, 2001:db8::257
--------	---------------------------

Security Groups

default	ALLOW IPv6 to ::/0 ALLOW IPv6 from default
---------	---

flavourinst

Overview	Interfaces	Log	Console	Action Log
----------	------------	-----	---------	------------

Name	flavourinst
ID	b5515a57-6f7c-4092-b829-7724ff78a906
Description	-
Project ID	55dc92c4ddfd4ba6b4c99b5353d7fd51
Status	Active
Locked	No
Availability Zone	nova
Created	Sept. 11, 2026, 1:23 p.m.
Age	5 minutes
Host	harsh
Instance Name	instance-0000001a
Reservation ID	r-xjg8yd0t
Launch Index	-
Hostname	flavourinst
Kernel ID	-
Ramdisk ID	-
Device Name	/dev/vda
User Data	-

Specs

Flavor Name	m1.nano
Flavor ID	42
RAM	192MB
vCPUs	1 vCPU
Disk	1GB

IP Addresses

public	172.24.4.231, 2001:db8::31b
--------	-----------------------------

Security Groups

default	ALLOW IPv6 to ::/0
---------	--------------------

Resize Instance



Flavor Choice *

[Advanced Options](#)

Old Flavor

m1.nano

New Flavor * ?

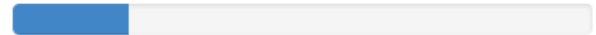
m1.micro

Flavor Details

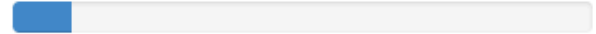
Name	m1.micro
vCPUs	1
Root Disk	1 GB
Ephemeral Disk	0 GB
Total Disk	1 GB
RAM	256 MB

Project Limits

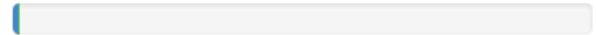
Number of Instances 2 of 10 Used



Number of vCPUs 2 of 20 Used



Total RAM 448 of 51,200 MB Used



Cancel

Resize

Instances

Instance ID ▾

Filter

Launch Instance

Delete Instances

More Actions ▾

Displaying 2 items

☐	Instance Name	Image Name	IP Address	Flavor	Key Pair	Status	Availability Zone	Task	Power State	Age	Actions
☐	flavourinst2	ciros-0.6.3-x86_64-disk	172.24.4.2, 2001:db8::257	m1.micro	-	Active	nova	None	Running	6 minutes	Create Snapshot ▾
☐	flavourinst	ciros-0.6.3-x86_64-disk	172.24.4.231, 2001:db8::31b	m1.micro	-	Active	nova	None	Running	7 minutes	Create Snapshot ▾

Displaying 2 items

Q4) create 20GB volume attach it to instance , create a snapshot of the volume ,create a new volume from that snapshot and verify the state

Get HPC as instance make attach volume to it of 20 GB and make snapshot of that volume and check it through the console of HPC

Instances

Instance ID Filter [Launch Instance](#) [Delete Instances](#) [More Actions](#)

Displaying 4 items

☐	Instance Name	Image Name	IP Address	Flavor	Key Pair	Status	Availability Zone	Task	Power State	Age	Actions
☐	ironman	cirros-0.6.3-x86_64-disk	172.24.4.234, 2001:db8::28	m1.nano	-	Active	nova	None	Running	6 minutes	Create Snapshot
☐	hpc	cirros-0.6.3-x86_64-disk	172.24.4.110, 2001:db8::22f	m1.micro	-	Active	nova	None	Running	30 minutes	Create Snapshot
☐	flavourinst2	cirros-0.6.3-x86_64-disk	172.24.4.2, 2001:db8::257	m1.micro	-	Active	nova	None	Running	59 minutes	Create Snapshot
☐	flavourinst	cirros-0.6.3-x86_64-disk	172.24.4.231, 2001:db8::31b	m1.micro	-	Active	nova	None	Running	59 minutes	Create Snapshot

Displaying 4 items

Went to HPC Console and perform the following Operations

```

login as 'cirros' user. default password: 'gocubsgo'. use 'sudo' for root.
cirros login: cirros
Password:
$ sudo su
$ fdisk -l
Disk /dev/vda: 1 GiB, 1073741824 bytes, 2097152 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: 62EC0A1C-A2E1-4BBD-9A38-AEF30C677F9C

Device      Start      End Sectors  Size Type
/dev/vda1  18432  2097118  2078687  1015M Linux filesystem
/dev/vda15  2048    18431   16384     8M EFI System

Partition table entries are not in disk order.

Disk /dev/vdb: 20 GiB, 21474836480 bytes, 41943040 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
$ mkfs.ext4 /dev/vdb
mke2fs 1.46.5 (30-Dec-2021)
Discarding device blocks: done
Creating filesystem with 5242880 4k blocks and 1310720 inodes
Filesystem UUID: d43ca380-6971-4f49-9c72-8a4cf4e8acc9
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208,
    4096000

Allocating group tables: done
Writing inode tables: done
Creating journal (32768 blocks): done
Writing superblocks and filesystem accounting information: done

$ mkdir /examdir
$ ls
$ mount /dev/vdb/examdir
mount: can't find /dev/vdb/examdir in /etc/fstab
$ mount /dev/vdb /examdir
$ cd /examdir
$ cat>test
Hello Harsh Welcome to ACC-HPC AI
^C
$ cat test
Hello Harsh Welcome to ACC-HPC AI
$

```

Then come to the instance and firstly detach the volume allocated to the old instance and Attach the same volume to the new Instance , go to the same console of that instance and perform the following operations.

```
[ 3.737091] x86/mm: Checked W+X mappings: passed, no W+X pages found.
[ 3.737831] x86/mm: Checking user space page tables
[ 3.883504] x86/mm: Checked W+X mappings: passed, no W+X pages found.
[ 3.884028] Run /init as init process

further output written to /dev/ttyS0
[ 4.637835] virtio_blk virtio2: [vdal 2097152 512-byte logical blocks (1.07 G
B/1.00 GiB)
[ 4.659350] GPT:Primary header thinks Alt. header is not at the end of the di
sk.
[ 4.659685] GPT:229375 != 2097151
[ 4.659902] GPT:Alternate GPT header not at the end of the disk.
[ 4.660126] GPT:229375 != 2097151
[ 4.660379] GPT: Use GNU Parted to correct GPT errors.
[ 4.941247] virtio_gpu virtio0: [drm] drm_plane_enable_fb_damage_clips() not called
[ 4.990505] random: crng init done

login as 'cirros' user, default password: 'gocubsgo'. use 'sudo' for root.
cirros login: [ 1598.462088] virtio_blk virtio5: [vdb1 41943040 512-byte logical blocks (21.5 GB/20.0 GiB)

login as 'cirros' user, default password: 'gocubsgo'. use 'sudo' for root.
cirros login: cirros
Password:
$ sudo su
$ fdisk -l
Disk /dev/vda: 1 GiB, 1073741824 bytes, 2097152 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: 62EC0A1C-A2E1-4BBD-9A3B-AEF30C677F9C

Device      Start      End Sectors  Size Type
/dev/vda1   18432    2097118  2078687  1015M Linux filesystem
/dev/vda15  2048      18431    16384      8M EFI System

Partition table entries are not in disk order.

Disk /dev/vdb: 20 GiB, 21474836480 bytes, 41943040 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
$ mkdir /testdir
$ mount /dev/vdb /testdir
$ cd /testdir
$ ls
lost+found test
$
```

Q5) launch instance , create a instance snapshot delete the original instance and recreate instance from that snapshot

Create Instance from launch instance (ironman1)

Instances

Instance ID =

Filter

Launch Instance

Delete Instances

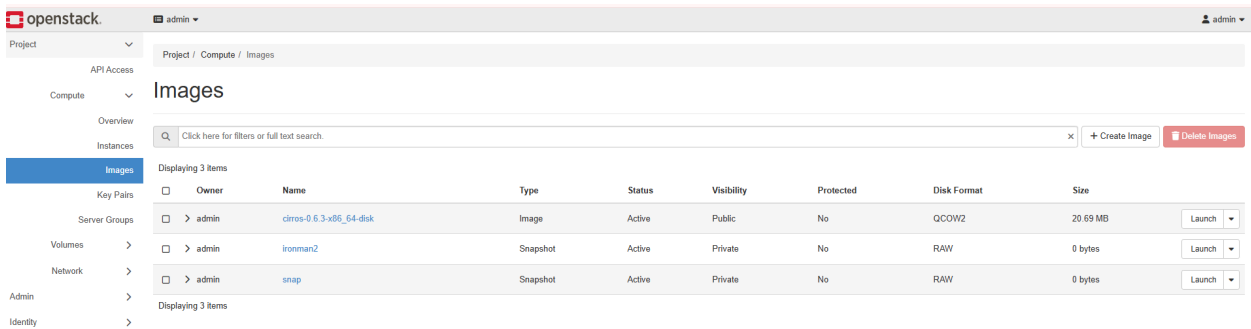
More Actions

Displaying 4 Items

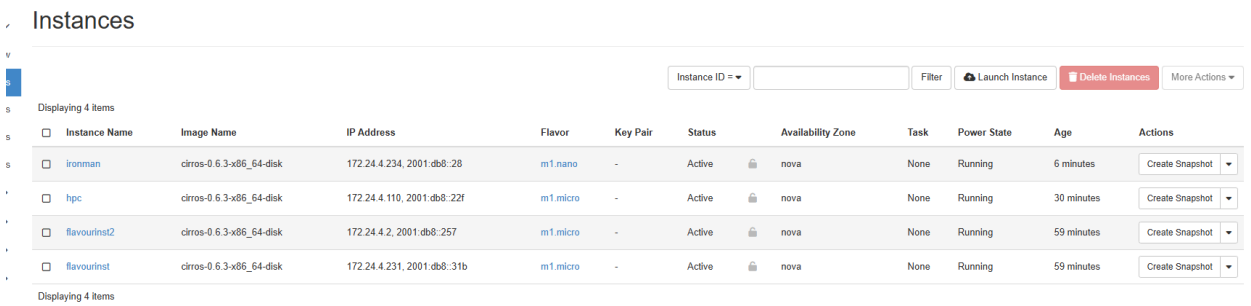
	Instance Name	Image Name	IP Address	Flavor	Key Pair	Status	Availability Zone	Task	Power State	Age	Actions
☐	ironman	cirros-0.6.3-x86_64-disk	172.24.4.234, 2001:db8::28	m1.nano	-	Active	nova	None	Running	2 minutes	Create Snapshot
☐	hpc	cirros-0.6.3-x86_64-disk	172.24.4.110, 2001:db8::22f	m1.micro	-	Active	nova	None	Running	26 minutes	Create Snapshot
☐	flavourinst2	cirros-0.6.3-x86_64-disk	172.24.4.2, 2001:db8::257	m1.micro	-	Active	nova	None	Running	55 minutes	Create Snapshot
☐	flavourinst	cirros-0.6.3-x86_64-disk	172.24.4.231, 2001:db8::31b	m1.micro	-	Active	nova	None	Running	55 minutes	Create Snapshot

Displaying 4 Items

Create Snapshot of that instance (ironman2) :



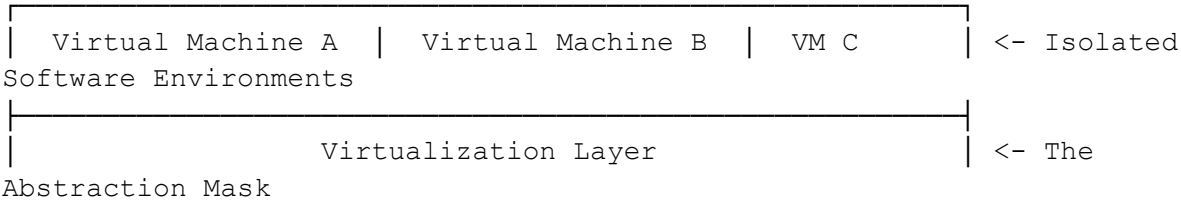
Delete the instance ironman and go to
The snapshot of that instance ironman2
Then launch the instance from ironman 2

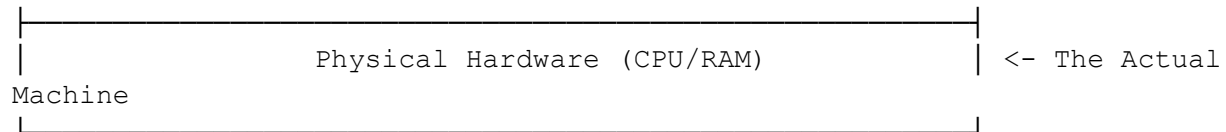


Q6.Explain The Termonologies

1. Virtualization: The Foundation of Cloud Computing

Virtualization abstracts physical computer hardware using a software layer, dividing one physical server into multiple distinct, secure, and isolated virtual systems.



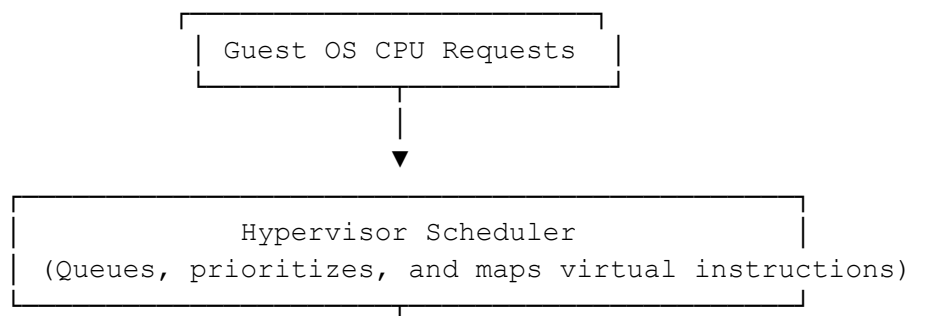


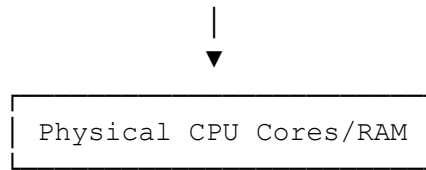
How it Works at a Low Level

- **Emulation & Abstraction:** CPU architectures include execution rings determining privilege levels. The host operating system runs at the highest privilege (Ring 0), direct control over hardware. Virtualization intercepts execution calls from "guest" systems via a specialized instruction set extension (like Intel VT-x or AMD-V).
 - **Partitioning:** The physical RAM is split using Nested Page Tables (NPT) or Extended Page Tables (EPT). The virtualized layer maps the guest operating system's perceived "physical" addresses to actual hardware addresses.
 - **Isolation:** If a virtual machine undergoes a system crash or kernel panic, it cannot access or corrupt the memory space of adjacent virtual systems or the host system.
-

2. Hypervisor: The Hardware Orchestrator

The **hypervisor** manages computing resources, serving as the traffic controller that schedules instructions from virtual environments onto physical processors.





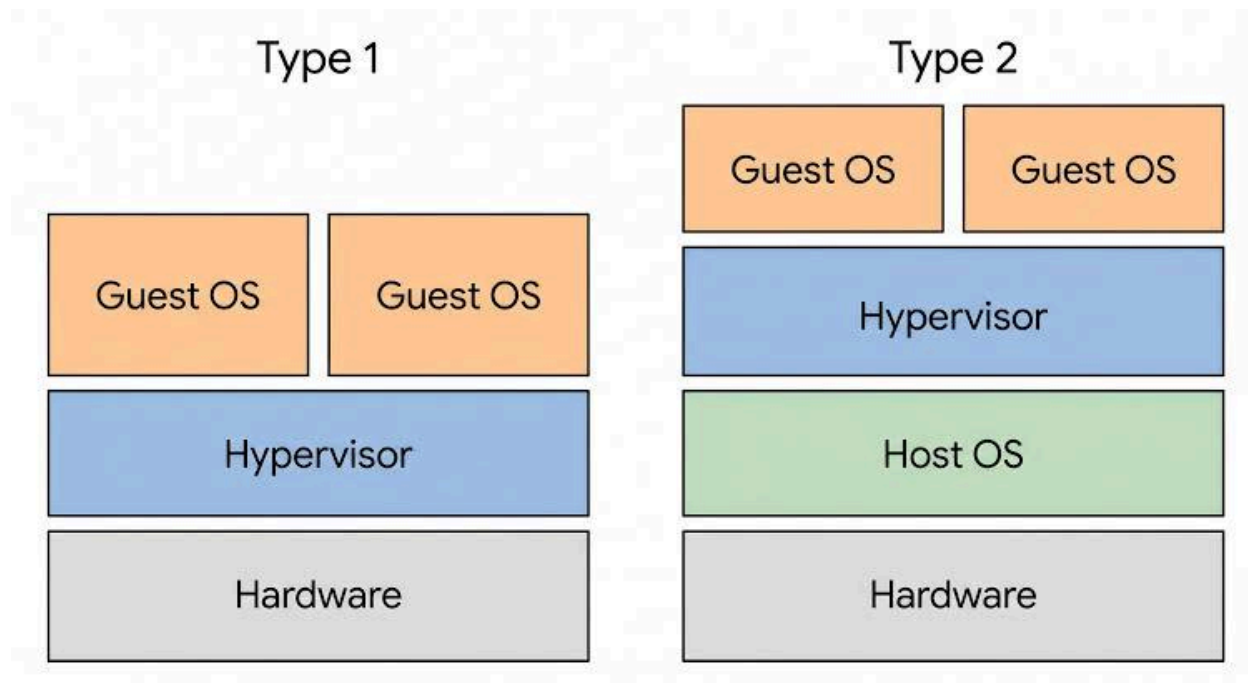
Core Responsibilities

- **CPU Scheduling:** Emulates virtual CPUs (vCPUs). A physical CPU thread handles time-sliced execution cycles across multiple vCPUs.
 - **Memory Management:** Allocates dynamic RAM limits, using techniques like memory ballooning to reclaim unused memory from one idle virtual machine and shift it to another under load.
 - **I/O Virtualization:** Intercepts disk storage reads/writes and network package demands, passing them down safely to the real physical controllers.
-

3. Type 1 vs. Type 2 Hypervisors: Architectural Differences

The separation line between hypervisor styles depends strictly on **where the management software is installed relative to the bare metal**.

Expand image



Use arrow keys to adjust value.

0:00/0:56

Made with Google AI

Type 1: Bare-Metal Hypervisors

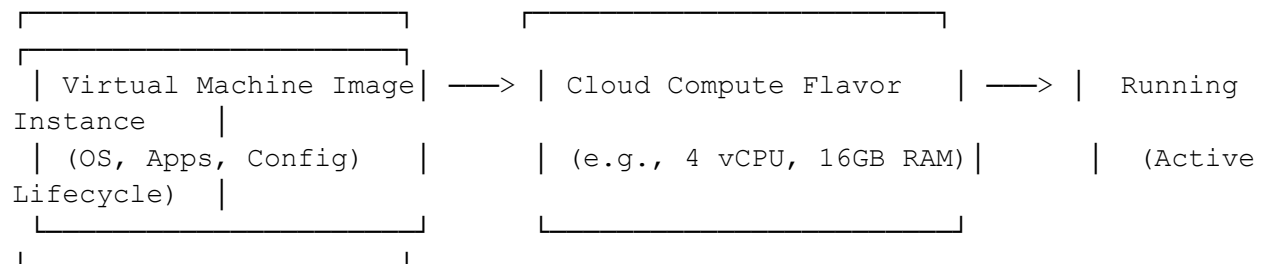
- **Architecture:** Type 1 options install **directly onto the hardware components**, serving as the base operating system itself.
- **Execution Paths:** Because there is no general-purpose host OS beneath it, context switching overhead is reduced. System instructions pass almost directly to the physical processor.
- **Enterprise Use:** Used in enterprise operations like AWS EC2 instances or internal corporate infrastructure via platforms like **VMware ESXi**, **Proxmox VE**, or **Microsoft Hyper-V**.

Type 2: Hosted Hypervisors

- **Architecture:** Type 2 engines run **nested as an application** inside a standard host operating system (e.g., Windows, macOS, Linux).
 - **Execution Paths:** A command must navigate from the guest virtual machine, through the hypervisor, down through the host OS kernel drivers, and finally hit the hardware. This extra layer increases latency.
 - **Development Use:** Ideal for software evaluation, local engineering environments, and desktop cross-compatibility using tools like **Oracle VirtualBox** or **VMware Workstation**.
-

4. Instance: The Provisioned Compute Node

An **instance** is the operational lifecycle unit of a virtual machine deployed within cloud infrastructure. It represents a structured bundle of dedicated virtual resources derived from a static template.



Lifecycle Anatomy

1. **The Base Image:** A pristine, read-only template file containing an operational kernel, core OS configurations, and target file architectures (such as an AML in AWS).

2. **The Instance Flavor/Type:** A declaration definition matching standard hardware parameters (e.g., compute-optimized, memory-heavy).
 3. **The Active State:** Once initiated, the cloud controller provisions memory space, establishes network access cards, ties storage mount pathways, and triggers the hypervisor boot sequence.
-

5. Instance Migration: Zero-Downtime Teleportation

Instance migration shifts an active, running virtual entity from an initial physical host hardware node to an entirely alternate target node without severing application connectivity.

Deep-Dive Mechanics of "Live Migration"

- **Pre-Copy Phase:** The active instance stays functional on the source host. The management system copies memory pages from the source server to the destination server.
 - **Iterative Dirty-Page Tracking:** While copying, the running instance continues modifying its memory. These changed areas ("dirty pages") are flagged and copied over in successive, smaller waves.
 - **Final Cutover (The Pivot):** When the remaining uncopied memory is minimal, the source hypervisor briefly pauses the instance (typically for a fraction of a second). The final dirty pages and the precise CPU register states are transferred.
 - **Network Announcement:** The destination host issues a Gratuitous ARP (Address Resolution Protocol) packet to the physical network switches. This informs the network that the instance's MAC address is now tied to a new physical switch port, routing live traffic to the new server seamlessly.
-

6. Boot From Volume: Externalizing the OS Root

In standard architectures, an instance boots from local ephemeral drives attached directly to the host server. **Boot from volume** decouples this mechanism by launching the instance from a remote block system connected over a fast storage network.

Technical Advantages

- **Persistent Lifecycles:** If the underlying physical compute host encounters a hardware crash, the virtual instance can be immediately re-spun on a different host server. It simply maps back to the exact same network storage volume, preventing data loss.
 - **Instantaneous Rescaling:** Because the entire operating system, configuration states, and log files live on an external drive, users can change instance types (e.g., moving from 2 cores to 32 cores) by detaching the volume and attaching it to a larger compute flavor.
-

7. Subnet: Logical Network Segmentation

A **subnet** divides a larger network range (expressed as a Classless Inter-Domain Routing or CIDR block) into isolated, smaller network routing components.

Network Design Architecture

- **CIDR Masking:** A network block like `10.0.0.0/16` provides 65,536 distinct IP paths. Breaking it down into `/24` subnets splits the pool into organized slices of 256 IPs each (e.g., `10.0.1.0/24`, `10.0.2.0/24`).
- **Routing Table Isolation:**
 - **Public Subnets:** The local subnet routing table includes an active entry directing external bound traffic out to an Internet Gateway edge device.
 - **Private Subnets:** Isolated from direct internet access. These subnets only route traffic internally or through a secure NAT (Network Address Translation) gateway, protecting databases and backend logic layers from external exposure.
-

8. Block vs. Object Access Management: Security Controls

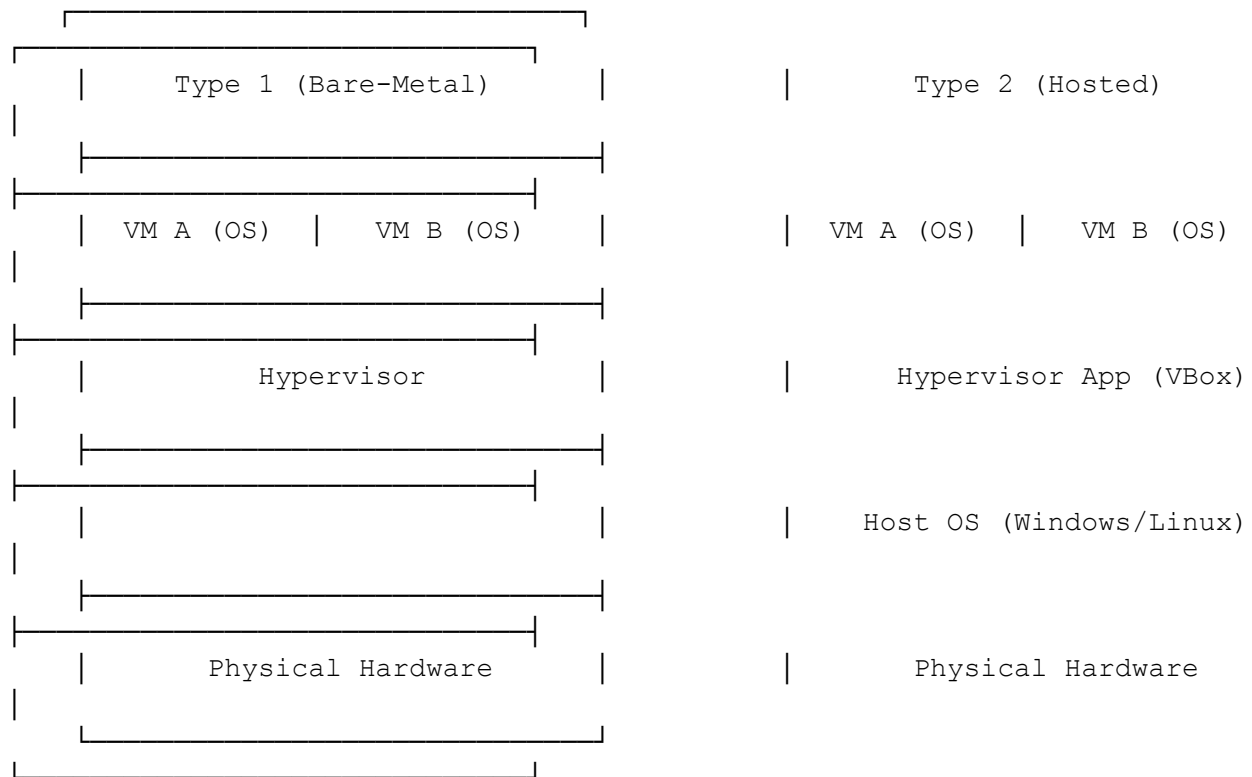
Data lifecycle control relies on two distinct structural architectures: **Block Storage** and **Object Storage**. They handle permissions, visibility, and data tracking differently.

Feature	Block Storage Access Management	Object Storage Access Management
Data Structure	Raw, unindexed blocks (fixed sectors without native metadata context).	Unstructured files wrapped with deep metadata inside a flat namespace bucket.
Primary Protocols	Fiber Channel, iSCSI, NVMe-oF (Network Fabrics).	HTTP/HTTPS REST APIs (e.g., GET, PUT, DELETE requests).
Access Control Mechanism	Managed via OS-level access lists, POSIX permissions, or storage network masking.	Managed via cloud Identity and Access Management , bucket policies, and presigned URLs.
Granularity	Coarse: Permissions are applied to the entire volume structure at the system layer.	Hyper-granular: Security rules can evaluate specific user attributes, target prefixes, or metadata values.

Would you like to explore a practical configuration example for these components, such as setting up a **subnet routing table architecture** or writing an **Object Storage bucket access policy**? Let me know which area you want to focus on next!

9.Type 1 And Type 2 Architectural Differences

The fundamental difference between the two hypervisor models is the **number of software layers between the virtual machine's operating system (Guest OS) and the actual physical hardware**.



1. Type 1 Hypervisor (Bare-Metal)

A Type 1 hypervisor acts as the base operating system for the machine it runs on. It is installed **directly onto the physical hardware** of a server.

Technical Execution

- **Direct Hardware Access:** When a guest virtual machine needs to execute a CPU instruction or write to memory, the Type 1 hypervisor coordinates that request directly with the physical motherboard, CPU, and RAM.

- **Minimal Footprint:** Because it does not need to support everyday desktop applications, web browsers, or graphical user interfaces (GUIs), the hypervisor software itself is lightweight—often only a few hundred megabytes.
 - **Kernel-Level Integration:** It usually runs in the processor's most privileged execution mode (Ring 0), giving it absolute control over scheduling hardware cycles.
-

2. Type 2 Hypervisor (Hosted)

A Type 2 hypervisor runs as an **application inside a standard host operating system** (like Windows 11, macOS, or Ubuntu Linux).

Technical Execution

- **The Layered Journey:** When a virtual machine makes a hardware request, that request must pass through the guest OS, hit the hypervisor application, get translated into a system call for the host OS kernel, and then pass down to the hardware drivers.
 - **Resource Reliance:** The hypervisor doesn't own the hardware; it must request RAM and CPU cycles from the host operating system's built-in scheduler. If your host OS decides to run a virus scan or a massive software update, the
-

Q7.Scenerio Based Question (Performed on Demo)

A] An Instance is Active but you can't reach the Instance.

B] Why The Instance can't be Deleted from the Horizon

C] How do you Differentiate Between Flavour and Resources

D] What is Talant

E] When you launch the Instance Process happening in the Background

D] What will happen the Instance is Deleted what will happen to it volume

A] An Instance is ACTIVE but you cannot reach it

If the OpenStack Horizon Dashboard shows an instance state as `ACTIVE`, it means the underlying hypervisor has successfully allocated hardware and booted the virtual machine kernel. If you still cannot connect via SSH, RDP, or Ping, the root cause is usually network or firewall misconfiguration:

1. **Missing Security Group Rules:** By default, OpenStack blocks all inbound traffic. If you haven't explicitly added an **ingress rule** for **Port 22 (SSH)**, **Port 3389 (RDP)**, or **ICMP (Ping)**, network requests will be dropped at the cloud boundary.
 2. **No Floating IP Assigned:** The instance might only have a private IP inside an isolated virtual network. To access it from your external machine, you must associate a **Floating IP** (publicly routable IP).
 3. **Key-Pair Missing or Incorrect:** If you are trying to use SSH but didn't inject a public OpenStack Keypair during creation, or if you are using the wrong private key file, the instance will reject the connection handshake.
 4. **OS Routing / Boot Failure:** The virtual hardware is up, but the guest operating system might be stuck in a boot loop, filesystem check, or configured with an incorrect internal static routing configuration. *Diagnostic Fix: Check the instance's console tab in Horizon to see what the OS screen is displaying.*
-

B] Why an Instance cannot be Deleted from Horizon

When you click "Delete Instance" and it fails or stays stuck, it is usually tied to state conflicts or backend API locking:

1. **Instance is Locked:** OpenStack allows administrators or owners to **Lock** an instance. When locked, any destructive action like terminating or deleting is strictly forbidden until an explicit "Unlock" command is triggered.
 2. **Stuck in a Transient State:** If the instance is stuck mid-process, the deletion command will fail. The backend Nova compute node expects the instance to be in a stable state before cleanup can initiate.
 3. **Cinder Volume Disconnection Failures:** If the instance is tied to an external storage volume and the underlying Storage Network (Cinder/Ceph) fails to safely unmount or detach that storage drive, the compute driver halts the deletion workflow to prevent data corruption.
-

C] Differentiating Between Flavor and Resources

- **Flavor:** A **Flavor** is a structural, predefined *template* or *profile* in OpenStack. It defines the specific allocation limits for a machine, such as the exact number of vCPUs, the total capacity of allowed RAM, and the size of the root disk. It is simply a recipe blueprint.

- **Resources: Resources** are the *actual, real-time physical or virtual capacities* consumed or available inside the cluster. While a flavor specifies that an instance *needs* 4 vCPUs and 8GB RAM, the "Resources" represent the active consumption pulled directly from the physical compute nodes, hypervisors, and storage pools.

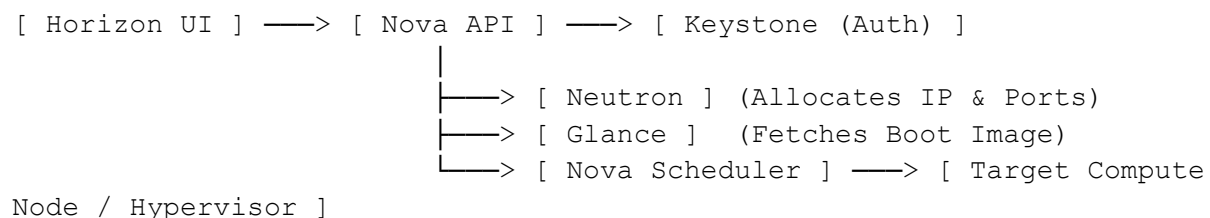
D] What is Tenant? (Note: Spelled "*Talant*" in the query)

In cloud infrastructure, **Tenant** (more commonly referred to as a **Project** in modern OpenStack versions) is the core logical unit of isolation and multi-tenancy.

- A Tenant acts as an isolated, virtual container that groups cloud resources together.
- Users, network subnets, computer instances, block volumes, and security policies are all tied to a specific tenant. Users assigned to "Tenant A" cannot view, modify, or interact with resources allocated inside "Tenant B" unless cross-tenant resource sharing is explicitly enabled by an administrator.

E] The Background Process When You Launch an Instance

When you click **Launch Instance** in Horizon, a highly orchestrated sequence occurs across the internal OpenStack components:



1. **Horizon to Nova API:** Horizon sends a REST API request to the compute controller .
2. **Authentication Check :** The system verifies your credentials, role privileges, and checks if your tenant has enough quota headroom left.
3. **Network Allocation):** The network component provisions virtual ports, hooks up security groups, and locks down an internal IP address for the instance.

4. **Image Retrieval** : The image service finds the requested OS template image. If booting from a volume, `Cinder` is triggered to clone that image into a fresh storage volume.
 5. **Placement Strategy** : The scheduler evaluates all physical servers in the datacenter to find a host hypervisor node that possesses enough unallocated vCPU and RAM capacity to fulfill the requested Flavor.
 6. **Hypervisor Provisioning**: The chosen computer host takes the configuration profile, pulls down a local cache of the OS image, creates the XML definitions, and triggers the hypervisor engine (like KVM or ESXi) to spin up the virtual machine.
-

F] What Happens to a Volume When Its Instance is Deleted?

The outcome depends strictly on the **Delete Volume on Instance Delete** configuration setting defined during the initial launch phase of that instance:

- **Scenario 1: Option Checked** — If this flag was enabled, OpenStack automatically purges the attached root storage volume from the backend system storage cluster (e.g., Cinder or Ceph) the moment the instance is terminated. **Data is completely lost.**
- **Scenario 2: Option Unchecked** — This is the default cloud behavior. When the instance is destroyed, only the compute runtime (CPU and RAM allocation) is wiped out. The storage volume detaches gracefully, remains fully intact, and reverts to an Available state. You can later attach this persistent volume to a completely different instance to retrieve your files.